Bài toán

- Dự đoán giá nhà.
- Dữ liệu gồm thông tin căn nhà và giá

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm

from sklearn.model_selection import learning_curve
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline

from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor
%matplotlib inline
```

In [3]:
```
dataset = pd.read_csv('kc_house_data.csv')
dataset
```

Out[3]:

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080
...	...	...	...	...	...	...	...
21608	263000018	20140521T000000	360000.0	3	2.50	1530	1131
21609	6600060120	20150223T000000	400000.0	4	2.50	2310	5813
21610	1523300141	20140623T000000	402101.0	2	0.75	1020	1350
21611	291310100	20150116T000000	400000.0	3	2.50	1600	2388
21612	1523300157	20141015T000000	325000.0	2	0.75	1020	1076

21613 rows × 21 columns

In [4]:
```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21613 entries, 0 to 21612
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   id             21613 non-null  int64  
 1   date           21613 non-null  object 
 2   price          21613 non-null  float64
 3   bedrooms       21613 non-null  int64  
 4   bathrooms      21613 non-null  float64
 5   sqft_living    21613 non-null  int64  
 6   sqft_lot       21613 non-null  int64  
 7   floors         21613 non-null  float64
 8   waterfront     21613 non-null  int64  
 9   view           21613 non-null  int64  
 10  condition      21613 non-null  int64  
 11  grade          21613 non-null  int64  
 12  sqft_above     21613 non-null  int64  
 13  sqft_basement  21613 non-null  int64  
 14  yr_built       21613 non-null  int64  
 15  yr_renovated   21613 non-null  int64  
 16  zipcode        21613 non-null  int64  
 17  lat            21613 non-null  float64
 18  long           21613 non-null  float64
 19  sqft_living15  21613 non-null  int64  
 20  sqft_lot15     21613 non-null  int64  
dtypes: float64(5), int64(15), object(1)
memory usage: 3.5+ MB
```

In [5]:
```
dataset['date'][0]
```

Out[5]:
```
'20141013T000000'
```

In [6]:
```
dataset.describe()
```

Out[6]:

	id	price	bedrooms	bathrooms	sqft_living	sqft_lo
count	2.161300e+04	2.161300e+04	21613.000000	21613.000000	21613.000000	2.161300e+0
mean	4.580302e+09	5.400881e+05	3.370842	2.114757	2079.899736	1.510697e+0
std	2.876566e+09	3.671272e+05	0.930062	0.770163	918.440897	4.142051e+0
min	1.000102e+06	7.500000e+04	0.000000	0.000000	290.000000	5.200000e+0
25%	2.123049e+09	3.219500e+05	3.000000	1.750000	1427.000000	5.040000e+0
50%	3.904930e+09	4.500000e+05	3.000000	2.250000	1910.000000	7.618000e+0
75%	7.308900e+09	6.450000e+05	4.000000	2.500000	2550.000000	1.068800e+0
max	9.900000e+09	7.700000e+06	33.000000	8.000000	13540.000000	1.651359e+0

In [7]:
```
X_data = dataset.drop(['price', 'id', 'date'], axis=1)
Y_data = dataset['price']
```

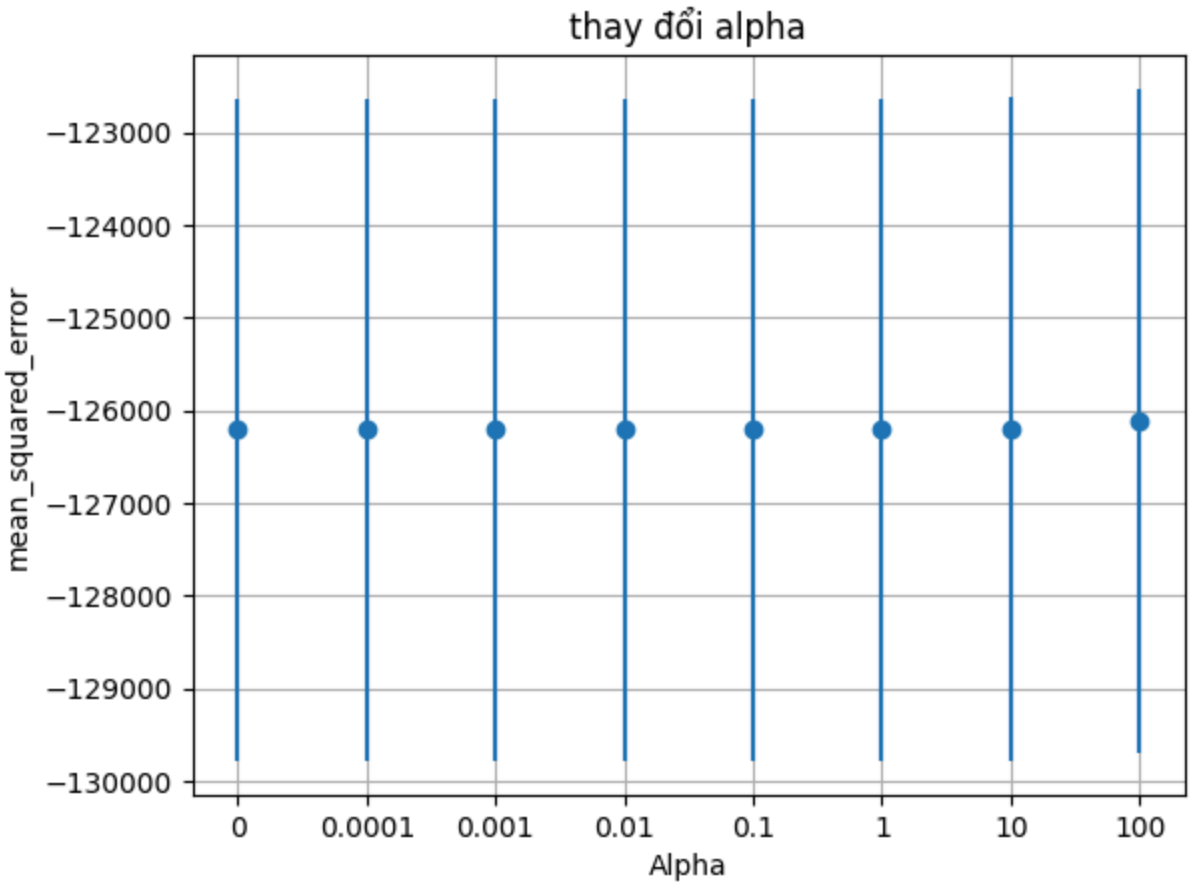
```
X_train, X_test, Y_train, Y_test = train_test_split(X_data, Y_data, test_size=0.2,
print("Dữ liệu training = ", X_train.shape, Y_train.shape)
print("Dữ liệu testing = ", X_test.shape, Y_test.shape)
```

```
Dữ liệu training =  (17290, 18) (17290,)
Dữ liệu testing =  (4323, 18) (4323,)
```

In [8]:
```
def cross_validation(estimator):
    _, train_scores, test_scores = learning_curve(estimator,
                                                  X_train, Y_train,
                                                  cv=10,
                                                  n_jobs=-1,
                                                  train_sizes=[1.0, ],
                                                  scoring='neg_mean_absolute_error'
    test_scores = test_scores[0]
    mean, std = test_scores.mean(), test_scores.std()
    return mean, std

def plot(title, xlabel, X, Y, error, ylabel = "mean_squared_error"):
    plt.xlabel(xlabel)
    plt.title(title)
    plt.grid()
    plt.ylabel(ylabel)

    plt.errorbar(X, Y, error, linestyle='None', marker='o')
```

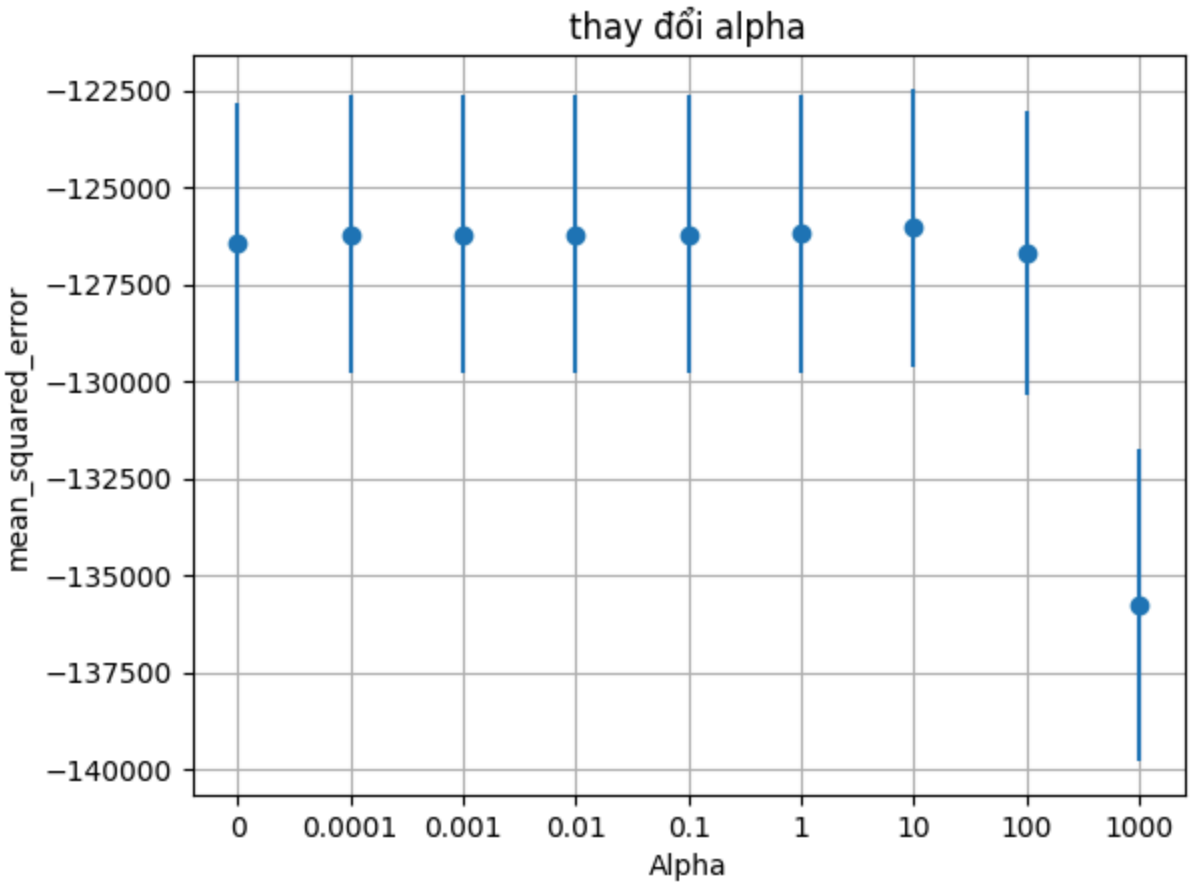
In [9]:
```
title = "thay đổi alpha"
xlabel = "Alpha"
X = []
Y = []
error = []

for aplha in tqdm([0, 0.0001, 0.001, 0.01, 0.1, 1, 10, 100]):
    text_clf = Lasso(alpha=aplha)
    mean, std = cross_validation(text_clf)
    X.append(str(aplha))
    Y.append(mean)
    error.append(std)

# lưu kết quả ra file ảnh
plot(title, xlabel, X, Y, error)
plt.show()
```

```
100%|██████████| 8/8 [00:10<00:00,  1.27s/it]
```



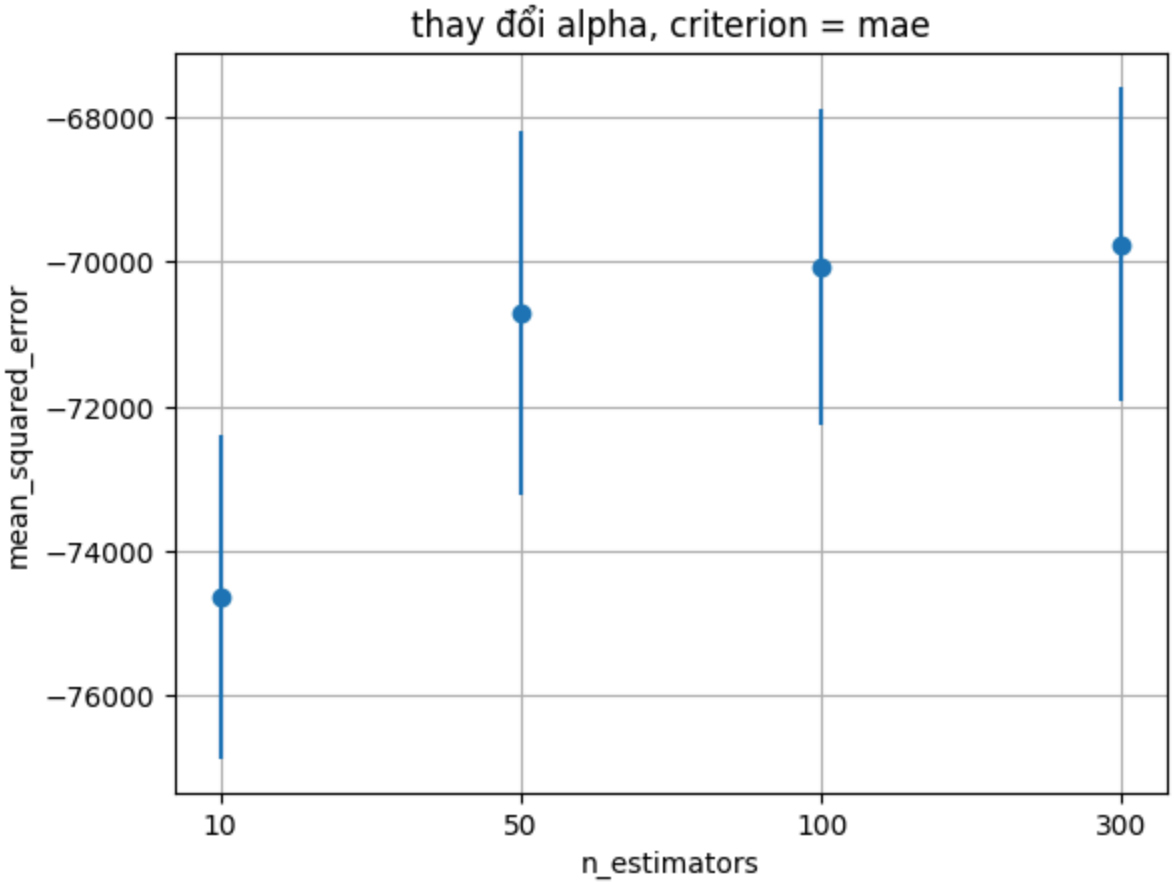
In [10]:
```
title = "thay đổi alpha"
xlabel = "Alpha"
X = []
Y = []
error = []

for aplha in tqdm([0, 0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000]):
    text_clf = Ridge(alpha=aplha)
    mean, std = cross_validation(text_clf)
    X.append(str(aplha))
    Y.append(mean)
    error.append(std)

# lưu kết quả ra file ảnh
plot(title, xlabel, X, Y, error)
plt.show()
```

```
100%|██████████| 9/9 [00:01<00:00,  4.71it/s]
```



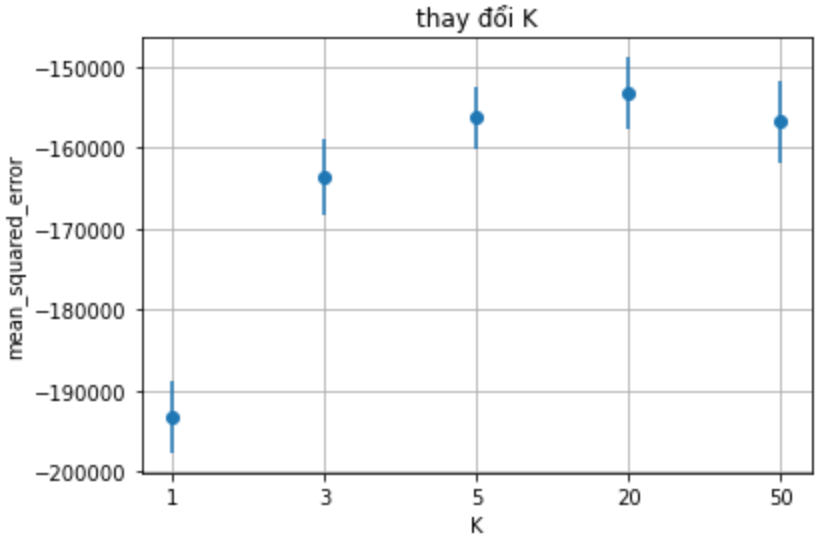
In [13]:
```
title = "thay đổi alpha, criterion = mae"
xlabel = "n_estimators"
X = []
Y = []
error = []

for n_estimators in tqdm([10, 50, 100, 300]):
    # Với từng giá trị n_estimators nhận được,
    # thực hiện xây dựng mô hình, huấn luyện và đánh giá theo cross-validation
    text_clf = RandomForestRegressor(criterion='absolute_error', n_estimators=n_est
    mean, std = cross_validation(text_clf)
    X.append(str(n_estimators))
    Y.append(mean)
    error.append(std)

# lưu kết quả ra file ảnh
plot(title, xlabel, X, Y, error)
# plt.savefig('images/RF_change_N.png', bbox_inches='tight')
plt.show()
```

```
100%|██████████| 4/4 [30:28<00:00, 457.05s/it]
```



In [18]:
```
title = "thay đổi K"
xlabel = "K"
X = []
Y = []
error = []

for k in tqdm([1, 3, 5, 20, 50]):
    # Với từng giá trị k nhận được,
    # thực hiện xây dựng mô hình, huấn luyện và đánh giá theo cross-validation
    text_clf = KNeighborsRegressor(n_neighbors=k)
    mean, std = cross_validation(text_clf)
    X.append(str(k))
    Y.append(mean)
    error.append(std)

# lưu kết quả ra file ảnh
plot(title, xlabel, X, Y, error)
plt.savefig('images/KNN_change_K.png', bbox_inches='tight')
plt.show()
```

```
100%|██████████| 5/5 [00:07<00:00,  1.46s/it]
```



In [14]:
```
# Kết quả dự đoán trên tập test
print(f'RF: {mean_absolute_error(Y_test, rf.predict(X_test))}')
print(f'KNN: {mean_absolute_error(Y_test, knn.predict(X_test))}')
print(f'Linear Regression: {mean_absolute_error(Y_test, lrg.predict(X_test))}')
print(f'Ridge: {mean_absolute_error(Y_test, ridge.predict(X_test))}')
print(f'Lasoo: {mean_absolute_error(Y_test, lasso.predict(X_test))}')
```

```
RF: 69948.35851615391
KNN: 158439.18634050427
Linear Regression: 125163.17629948513
Ridge: 125027.29664089366
Lasoo: 125090.93815064323
```

In [13]:
```
rf = RandomForestRegressor(criterion='mse', n_estimators=300)
knn = KNeighborsRegressor(n_neighbors=20)
lrg = LinearRegression()
ridge = Ridge(alpha=10)
lasso = Lasso(alpha=100)
# Huấn luyện các mô hình trên tập dữ liệu train đầy đủ
rf.fit(X_train, Y_train)
knn.fit(X_train, Y_train)
lrg.fit(X_train, Y_train)
ridge.fit(X_train, Y_train)
lasso.fit(X_train, Y_train)
```

```
C:\Users\Admin\anaconda3\lib\site-packages\sklearn\linear_model\_coordinate_descent.
py:531: ConvergenceWarning: Objective did not converge. You might want to increase t
he number of iterations. Duality gap: 319194110679868.75, tolerance: 232310058468.82
26
  positive)
```

Out[13]:
```
Lasso(alpha=100)
```